

Aula 12 - Introdução ao RAG (Retrieval-Augmented Generation)

Módulo: Básico de IA Generativa

Carga Horária: 3 horas (teoria e prática)

Autores: [A definir]

Desafio

Uma empresa de médio porte possui um manual de procedimentos internos com mais de 200 páginas, atualizado frequentemente. Colaboradores novos levam semanas para encontrar informações básicas, e mesmo os veteranos às vezes não sabem onde está documentado determinado procedimento. Quando perguntam ao ChatGPT, ele responde com informações genéricas que não refletem os procedimentos específicos da empresa — ou pior, inventa respostas que parecem corretas mas não são (as chamadas "alucinações").

A arquitetura RAG (Retrieval-Augmented Generation) resolve esse problema: antes de responder, o sistema busca trechos relevantes nos documentos internos e usa esse contexto para gerar uma resposta fundamentada. Assim, a IA responde com base no manual real da empresa, não em conhecimento genérico, e pode inclusive citar a fonte da informação.

Objetivos

- Compreender a arquitetura RAG e seus componentes
 - Explicar como o RAG reduz alucinações em respostas de LLMs
 - Aplicar estratégias de chunking para divisão de documentos
 - Avaliar a qualidade de sistemas RAG com métricas apropriadas
-

1. Fundamentos do RAG

1.1 O que é RAG?

O RAG (Retrieval-Augmented Generation) é uma arquitetura que integra **recuperação de informações** com **modelos de linguagem generativos**, permitindo que respostas sejam

produzidas com base em documentos externos ao modelo. Em vez de depender exclusivamente do conhecimento interno do LLM, o RAG consulta uma base documental previamente indexada e injeta os trechos relevantes como contexto para a geração da resposta.

Segundo revisões sistemáticas recentes, um pipeline RAG típico é composto por quatro estágios principais: **chunking**, **embedding**, **(re)ranking** e **geração**. Inicialmente, documentos extensos são segmentados em partes menores e semanticamente autocontidas. Em seguida, esses segmentos são convertidos em vetores de embedding, armazenados em um banco vetorial e recuperados conforme a similaridade com a pergunta do usuário. Por fim, o LLM gera a resposta utilizando exclusivamente o contexto recuperado (ARXIV, 2024).

1.2 Por que usar RAG?

O uso de RAG resolve limitações estruturais dos modelos de linguagem quando utilizados de forma isolada. LLMs puros operam com conhecimento estático e genérico, enquanto o RAG permite acesso a **informações atualizadas, específicas e verificáveis**, reduzindo riscos de alucinação e aumentando a confiabilidade das respostas.

Problema do LLM Puro	Solução com RAG
Conhecimento desatualizado	Busca em documentos atuais
Alucinações	Respostas baseadas em fontes
Conhecimento genérico	Contexto específico do domínio
Sem citação de fontes	Referências aos documentos

Pesquisas indicam ainda que inserir grandes volumes de texto diretamente na janela de contexto do LLM é uma estratégia ineficiente. Esse método de “força bruta” provoca o efeito conhecido como **Lost in the Middle**, no qual o modelo perde atenção sobre partes relevantes do conteúdo, degradando a qualidade da resposta (RAGFLOW, 2025). O RAG mitiga esse problema ao fornecer apenas os trechos mais relevantes no momento da geração.

2. Componentes do RAG

O funcionamento do RAG pode ser representado como um fluxo sequencial, no qual a pergunta do usuário é transformada em embedding, comparada com documentos indexados e enriquecida com contexto antes da geração da resposta.

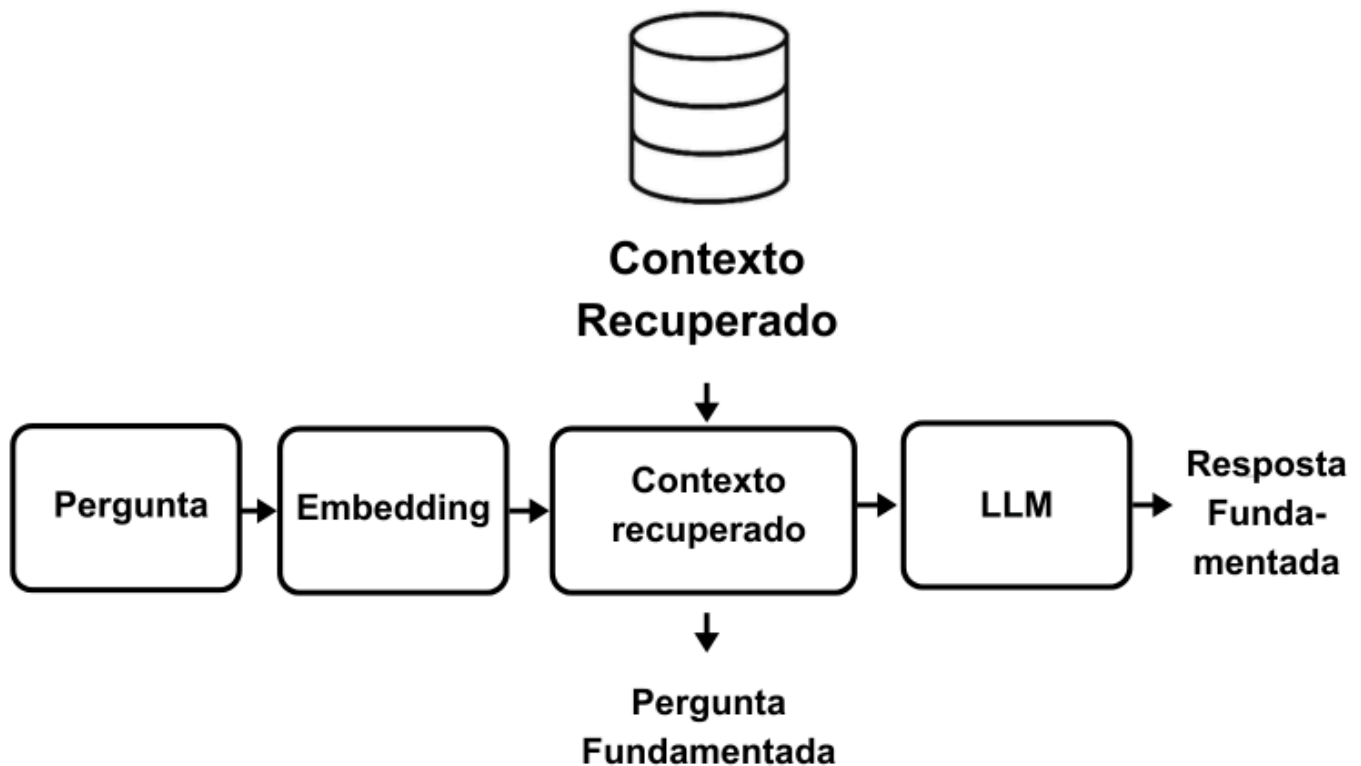


Figura 32: Arquitetura RAG - da pergunta à resposta fundamentada

O pipeline abaixo garante que o modelo gere respostas **ancoradas em dados reais**, reduzindo inferências não fundamentadas e permitindo maior controle sobre as fontes utilizadas.

2.1 Implementação Básica

No código abaixo, a classe `RAGBasico` implementa um **pipeline RAG funcional de ponta a ponta**, utilizando **ChromaDB como banco vetorial** e **embeddings da OpenAI**. No método `__init__`, o sistema inicializa um cliente OpenAI e configura uma coleção persistente no ChromaDB, associando automaticamente uma função de embedding baseada no modelo `text-embedding-3-small`.

O método `indexar` é responsável por inserir documentos na base vetorial, atribuindo identificadores únicos e metadados opcionais. Cada documento é automaticamente convertido em embedding e armazenado pelo ChromaDB. O método `recuperar` executa a etapa de **retrieval**, buscando os documentos semanticamente mais próximos da pergunta do usuário com base na similaridade vetorial.

O método `gerar_resposta` realiza a etapa de **generation**, formatando os documentos recuperados como contexto explícito e instruindo o modelo a responder **exclusivamente com base nessas fontes**, citando-as quando apropriado. Por fim, o método `consultar` orquestra todo o fluxo RAG, executando a recuperação do contexto e a geração da resposta em uma única chamada de alto nível.

Esse exemplo demonstra um RAG mínimo, porém completo, adequado para manuais corporativos, bases de conhecimento internas e sistemas de atendimento ao cliente, servindo como base para evoluções mais avançadas, como filtros por metadados, reranking e controle de confiança.

```
from openai import OpenAI
import chromadb
from chromadb.utils import embedding_functions
from dotenv import load_dotenv

load_dotenv()

class RAGBasico:
    """Sistema RAG básico para consulta de documentos."""

    def __init__(self, nome_colecao: str = "documentos"):
        self.client = OpenAI()

        # Configurar ChromaDB (em memória para teste)
        self.chroma = chromadb.Client()
        self.embedding_fn = embedding_functions.OpenAIEmbeddingFunction(
            model_name="text-embedding-3-small"
        )
        self.colecao = self.chroma.get_or_create_collection(
            name=nome_colecao,
            embedding_function=self.embedding_fn
        )

    def indexar(self, documentos: list, metadados: list = None):
        """Indexa documentos na base vetorial."""
        ids = [f"doc_{i}" for i in range(self.colecao.count(),
self.colecao.count() + len(documentos))]

        kwargs = {"documents": documentos, "ids": ids}
        if metadados:
            kwargs["metadatas"] = metadados

        self.colecao.add(**kwargs)

        print(f"Indexados {len(documentos)} documentos. Total:
{self.colecao.count()}")

    def recuperar(self, pergunta: str, k: int = 3) -> list:
        """Recupera documentos relevantes para a pergunta."""
        resultados = self.colecao.query(
```

```

        query_texts=[pergunta],
        n_results=k
    )

    return [
        {"texto": doc, "metadados": meta}
        for doc, meta in zip(
            resultados["documents"][0],
            resultados["metadatas"][0]
        )
    ]

def gerar_resposta(self, pergunta: str, contextos: list) -> str:
    """Gera resposta usando contexto recuperado."""
    # Formatar contexto
    contexto_formatado = "\n\n".join([
        f"[Fonte {i+1}]: {c['texto']}"
        for i, c in enumerate(contextos)
    ])

    prompt = f"""
    Responda à pergunta usando APENAS as informações fornecidas no
    contexto.
    Se a informação não estiver no contexto, diga que não encontrou essa
    informação nos documentos.
    Cite as fontes usadas na resposta.

    CONTEXTO:
    {contexto_formatado}

    PERGUNTA: {pergunta}

    RESPOSTA:
    """

    response = self.client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": "Você responde perguntas
baseando-se apenas no contexto fornecido."},
            {"role": "user", "content": prompt}
        ]
    )

    return response.choices[0].message.content

```

```

def consultar(self, pergunta: str, k: int = 3) -> dict:
    """Realiza consulta completa: recupera e gera resposta."""
    # 1. Recuperar contexto
    contextos = self.recuperar(pergunta, k)

    # 2. Gerar resposta
    resposta = self.gerar_resposta(pergunta, contextos)

    return {
        "pergunta": pergunta,
        "resposta": resposta,
        "fontes": contextos
    }

# Exemplo de uso
rag = RAGBasico("manual_procedimentos")

# Indexar manual de procedimentos
procedimentos = [
    "Para solicitar férias, o colaborador deve preencher o formulário F-001 com 30 dias de antecedência.",
    "O horário de expediente é das 8h às 18h, com tolerância de 15 minutos.",
    "Atestado médico superior a 3 dias requer envio ao RH em até 48 horas.",
    "A promoção por mérito ocorre a cada 2 anos de acordo com avaliação de desempenho.",
]

rag.indexar(procedimentos)

# Consultar
resultado = rag.consultar("Como faço para tirar férias?")
print(f"\nPergunta: {resultado['pergunta']}")
print(f"\nResposta: {resultado['resposta']}")
print(f"\nFontes utilizadas:")
for i, fonte in enumerate(resultado['fontes']):
    print(f"    [{i+1}] {fonte['texto'][:80]}...")

```

Execução: `python exemplo01.py`

3. Estratégias de Chunking

3.1 A Importância do Chunking

O chunking é um dos fatores mais críticos para o sucesso de um sistema RAG, pois define **como o conhecimento será fragmentado, indexado e posteriormente recuperado**. Uma estratégia de chunking inadequada pode resultar em contextos incompletos, respostas fragmentadas ou perda de informações essenciais durante a recuperação. Em contrapartida, um bom chunking produz unidades semanticamente coerentes, aumentando a precisão da busca vetorial e a qualidade da resposta gerada pelo LLM. Estudos recentes destacam que a escolha da estratégia de chunking influencia diretamente a relevância dos documentos recuperados e a consistência contextual das respostas (MEDIUM, 2025).

3.2 Tipos de Chunking

As estratégias de chunking variam conforme a **estrutura do documento**, o **domínio do conteúdo** e o **tipo de pergunta esperado**. Documentos normativos, por exemplo, se beneficiam de divisão por artigos ou seções, enquanto textos narrativos ou manuais operacionais tendem a funcionar melhor com chunking por tamanho fixo ou por sentenças.

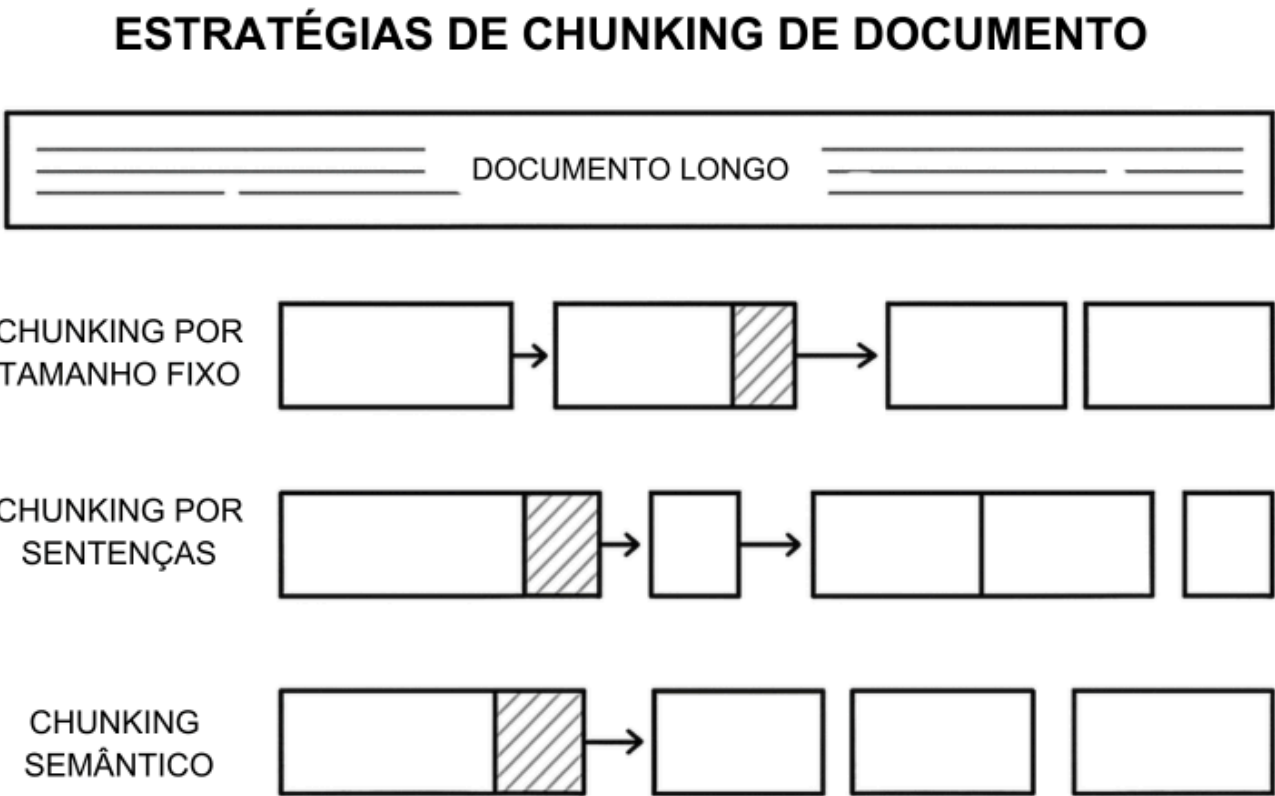


Figura 33: Diferentes estratégias de divisão de documentos

No código abaixo, a classe `Chunker` reúne múltiplas estratégias de chunking aplicáveis a documentos administrativos e jurídicos. O método `por_tamanho_fixo` divide o texto em blocos de tamanho constante, com sobreposição entre chunks, técnica útil para preservar continuidade semântica em textos longos. O método `por_sentencas` agrupa um número definido de sentenças em cada chunk, mantendo unidades de sentido mais naturais,

especialmente eficaz para textos discursivos. Já o método `por_paragrafos` utiliza quebras de parágrafo como critério de divisão, sendo indicado para relatórios e manuais bem estruturados.

Para documentos formais e normativos, o método `por_secoes` realiza a divisão com base em marcadores estruturais, como artigos, capítulos ou seções, preservando a hierarquia lógica do texto. Essa abordagem é especialmente adequada para legislações, decretos e regulamentos, pois mantém cada chunk alinhado a uma unidade normativa completa, facilitando a recuperação precisa de dispositivos legais específicos.

```
import re
from typing import List

class Chunker:
    """Estratégias de chunking para documentos."""

    @staticmethod
    def por_tamanho_fixo(texto: str, tamanho: int = 500, overlap: int = 50)
-> List[str]:
    """Divide texto em chunks de tamanho fixo com sobreposição."""
    chunks = []
    inicio = 0

    while inicio < len(texto):
        fim = inicio + tamanho
        chunk = texto[inicio:fim]
        chunks.append(chunk.strip())
        inicio = fim - overlap

    return chunks

    @staticmethod
    def por_sentencas(texto: str, sentencas_por_chunk: int = 5) ->
List[str]:
    """Divide texto por sentenças."""
    # Dividir em sentenças
    sentencas = re.split(r'(?<=[.!?])\s+', texto)

    # Agrupar sentenças
    chunks = []
    for i in range(0, len(sentencas), sentencas_por_chunk):
        chunk = ' '.join(sentencas[i:i + sentencas_por_chunk])
        if chunk.strip():
            chunks.append(chunk.strip())

    return chunks
```



```

@staticmethod
def por_paragrafos(texto: str) -> List[str]:
    """Divide texto por parágrafos."""
    paragrafos = texto.split('\n\n')
    return [p.strip() for p in paragrafos if p.strip()]

@staticmethod
def por_secoes(texto: str, marcador_secao: str =
r'^(?:Art\.|Capítulo|Seção)') -> List[dict]:
    """Divide texto por seções (para documentos estruturados)."""

    linhas = texto.split('\n')
    secoes = []
    secao_atual = {"titulo": "Introdução", "conteudo": []}

    for linha in linhas:
        if re.match(marcador_secao, linha.strip()):
            # Salvar seção anterior
            if secao_atual["conteudo"]:
                secao_atual["conteudo"] =
'\n'.join(secao_atual["conteudo"])
                secoes.append(secao_atual)
            # Iniciar nova seção
            secao_atual = {"titulo": linha.strip(), "conteudo": []}
        else:
            secao_atual["conteudo"].append(linha)

    # Adicionar última seção
    if secao_atual["conteudo"]:
        secao_atual["conteudo"] = '\n'.join(secao_atual["conteudo"])
        secoes.append(secao_atual)

    return secoes

# Exemplo de uso
texto_exemplo = """
Art. 1º – Este regulamento dispõe sobre os procedimentos internos da
empresa.

Art. 2º – Todo colaborador deve cumprir horário de expediente.
O horário padrão é das 8h às 18h.
Existe tolerância de 15 minutos para entrada e saída.

Art. 3º – As férias devem ser solicitadas com antecedência.
O prazo mínimo é de 30 dias antes do início do gozo.

```

```
.....
```

```
chunker = Chunker()

# Testar diferentes estratégias
print("=== Por tamanho fixo (100 chars, overlap 20) ===")
for i, chunk in enumerate(chunker.por_tamanho_fixo(texto_exemplo, 100, 20)):
    print(f"  Chunk {i+1}: {chunk[:60]}...")

print("\n=== Por sentenças (2 por chunk) ===")
for i, chunk in enumerate(chunker.por_sentencas(texto_exemplo, 2)):
    print(f"  Chunk {i+1}: {chunk[:60]}...")

print("\n=== Por parágrafos ===")
for i, chunk in enumerate(chunker.por_paragrafos(texto_exemplo)):
    print(f"  Chunk {i+1}: {chunk[:60]}...")

print("\n=== Por seções (artigos) ===")
for secao in chunker.por_secoes(texto_exemplo):
    print(f"  Seção: {secao['titulo']}")
    print(f"          {secao['conteudo'][:60]}...")
```

Execução: `python exemplo02.py`

3.3 Recomendações de Chunking

A escolha da estratégia de chunking deve considerar o tipo de documento, o tamanho médio dos textos e o nível de granularidade esperado nas respostas. A tabela a seguir resume recomendações práticas para diferentes categorias de documentos comumente utilizadas em sistemas RAG no ambiente corporativo.

Tipo de Documento	Estratégia Recomendada	Tamanho Sugerido
Legislação	Por artigos/seções	1 artigo por chunk
Manuais	Por seções + overlap	500-1000 chars
FAQs	Por pergunta-resposta	1 par por chunk
Relatórios	Por parágrafos	2-3 parágrafos

Requisitos

- Python 3.8+

- Bibliotecas: `openai`, `chromadb`

Instalação:

```
pip install openai chromadb python-dotenv
```

Atividade Prática

Criar uma aplicação que implementa um pipeline RAG básico: (1) carregar um documento de políticas internas da empresa, (2) dividir em chunks, (3) gerar embeddings e armazenar, (4) criar função que recebe pergunta, busca contexto relevante e gera resposta fundamentada. Use o Chroma DB que é gratuito.

Critérios de avaliação:

- ☐ Documento processado e indexado
- ☐ Chunking adequado implementado
- ☐ Busca semântica funcionando
- ☐ Respostas fundamentadas no contexto
- ☐ Citação de fontes

Referências

ARXIV. **A Systematic Review of Key Retrieval-Augmented Generation (RAG) Systems**. Disponível em: <https://arxiv.org/html/2507.18910v1>. Acesso em: jan. 2025.

ARXIV. **Retrieval-Augmented Generation: A Comprehensive Survey**. Disponível em: <https://arxiv.org/html/2506.00054v1>. Acesso em: jan. 2025.

DATANUCLEUS. **RAG in 2025: The enterprise guide**. Disponível em: <https://datanucleus.dev/rag-and-agentic-ai/what-is-rag-enterprise-guide-2025>. Acesso em: jan. 2025.

EDENAI. **The 2025 Guide to Retrieval-Augmented Generation (RAG)**. Disponível em: <https://www.edenai.co/post/the-2025-guide-to-retrieval-augmented-generation-rag>. Acesso em: jan. 2025.

MEDIUM. **Chunking Strategies for RAG: A Comprehensive Guide**. Disponível em: <https://medium.com/@adnanmasood/chunking-strategies-for-retrieval-augmented-generation->

[rag](#). Acesso em: jan. 2025.

RAGFLOW. **The Rise and Evolution of RAG in 2024**. Disponível em: <https://ragflow.io/blog/the-rise-and-evolution-of-rag-in-2024-a-year-in-review>. Acesso em: jan. 2025.

RAGFLOW. **From RAG to Context - A 2025 year-end review**. Disponível em: <https://ragflow.io/blog/rag-review-2025-from-rag-to-context>. Acesso em: jan. 2025.

SPRINGER. **Retrieval-Augmented Generation (RAG)**. Business & Information Systems Engineering. Disponível em: <https://link.springer.com/article/10.1007/s12599-025-00945-3>. Acesso em: jan. 2025.

Anexos

RAG Avançado

À medida que sistemas RAG evoluem para cenários reais de uso — como bases jurídicas, normativas ou administrativas — torna-se necessário ir além do pipeline básico de recuperação e geração. Um **RAG avançado** incorpora técnicas adicionais como **chunking configurável**, **reformulação de perguntas**, **recuperação híbrida com reranking**, **citações explícitas de fontes** e **suporte a contexto conversacional**. Esses mecanismos aumentam significativamente a precisão da recuperação, reduzem ambiguidades e tornam as respostas mais auditáveis e confiáveis, requisitos essenciais em ambientes corporativos e empresariais.

Sistema RAG Completo

No código abaixo, a classe `RAGAvancado` implementa um sistema RAG completo com múltiplas camadas de sofisticação. O método `processar_documento` realiza a ingestão de documentos aplicando diferentes estratégias de chunking (fixo, por sentenças, parágrafos ou seções), armazenando cada fragmento com metadados que identificam sua origem e estratégia utilizada. O método `reformular_pergunta` melhora a etapa de recuperação ao transformar perguntas dependentes de histórico em perguntas autocontidas, técnica fundamental para RAG conversacional.

O método `recuperar_hibrido` executa uma recuperação em duas etapas: primeiro realiza uma busca vetorial ampla e, em seguida, aplica **reranking com LLM**, reordenando os candidatos com base na relevância semântica real em relação à pergunta. Já o método `gerar_com_citacoes` gera respostas textuais contendo **citações inline**, permitindo rastrear exatamente quais documentos fundamentaram cada afirmação. Por fim, o método `consultar` orquestra todo o fluxo — reformulação, recuperação, reranking e geração — entregando uma resposta final robusta, explicável e adequada a contextos críticos.

```

from dataclasses import dataclass
from typing import List, Optional
import json

@dataclass
class DocumentoRAG:
    texto: str
    fonte: str
    pagina: Optional[int] = None
    secao: Optional[str] = None

class RAGAvancado:
    """Sistema RAG com funcionalidades avançadas."""

    def __init__(self, nome_base: str):
        self.client = OpenAI()
        self.chroma = chromadb.PersistentClient(path=f"./rag_{nome_base}")
        self.embedding_fn = embedding_functions.OpenAIEmbeddingFunction(
            model_name="text-embedding-3-small"
        )
        self.colecao = self.chroma.get_or_create_collection(
            name=nome_base,
            embedding_function=self.embedding_fn
        )
        self.chunker = Chunker()

    def processar_documento(self, texto: str, fonte: str,
                           estrategia: str = "sentencas") -> int:
        """
        Processa e indexa documento completo.

        Args:
            texto: Texto do documento
            fonte: Identificação da fonte
            estrategia: "fixo", "sentencas", "paragrafos", "secoes"

        Returns:
            Número de chunks criados
        """
        # Aplicar chunking
        if estrategia == "fixo":
            chunks = self.chunker.por_tamanho_fixo(texto, 500, 50)
        elif estrategia == "sentencas":
            chunks = self.chunker.por_sentencas(texto, 3)
        elif estrategia == "paragrafos":
            chunks = self.chunker.por_paragrafos(texto)

```

```

elif estrategia == "secoes":
    secoes = self.chunker.por_secoes(texto)
    chunks = [s["conteudo"] for s in secoes]
else:
    chunks = [texto]

# Preparar metadados
inicio = self.colecao.count()
ids = [f"chunk_{inicio + i}" for i in range(len(chunks))]
metadados = [
    {"fonte": fonte, "chunk_index": i, "estrategia": estrategia}
    for i in range(len(chunks))
]

# Indexar
self.colecao.add(
    documents=chunks,
    metadatas=metadados,
    ids=ids
)

return len(chunks)

def reformular_pergunta(self, pergunta: str, historico: list = None) ->
str:
    """
    Reformula pergunta para melhor recuperação.

    Args:
        pergunta: Pergunta original
        historico: Histórico de conversa

    Returns:
        Pergunta reformulada
    """
    if not historico:
        return pergunta

    contexto_conversa = "\n".join([
        f"{'Usuário' if i%2==0 else 'Assistente'}: {msg}"
        for i, msg in enumerate(historico[-4:])
    ])

    prompt = f"""
    Dado o histórico da conversa e a nova pergunta, reformule a pergunta
    para ser autocontida (sem depender do contexto anterior).

```

HISTÓRICO:

{contexto_conversa}

NOVA PERGUNTA: {pergunta}

PERGUNTA REFORMULADA:

"""

```
response = self.client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": prompt}],
    max_tokens=200
)
```

```
return response.choices[0].message.content.strip()
```

```
def recuperar_hibrido(self, pergunta: str, k: int = 5) -> list:
    """
```

Recuperação híbrida: semântica + reranking.

Args:

pergunta: Pergunta do usuário
k: Número final de resultados

Returns:

Documentos reranqueados

"""

```
# Busca inicial (mais resultados)
resultados_iniciais = self.colecao.query(
    query_texts=[pergunta],
    n_results=k * 2
)
```

```
# Reranking com LLM
```

```
candidatos = [
    {"texto": doc, "meta": meta}
    for doc, meta in zip(
        resultados_iniciais["documents"][0],
        resultados_iniciais["metadatas"][0]
    )
]
```

```
prompt_rerank = f"""
```

Dado a pergunta e os documentos candidatos, ordene os documentos do mais relevante ao menos relevante.

PERGUNTA: {pergunta}

DOCUMENTOS:

```
{json.dumps([{"id": i, "texto": c["texto"][:200]} for i, c in
enumerate(candidatos)], ensure_ascii=False)}
```

Retorne JSON com lista de IDs ordenados: {"ordem": [id1, id2, ...]}

```
response = self.client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": prompt_rerank}],
    response_format={"type": "json_object"}
)
```

```
ordem = json.loads(response.choices[0].message.content)["ordem"]
```

```
return [candidatos[i] for i in ordem[:k]]
```

```
def gerar_com_citacoes(self, pergunta: str, contextos: list) -> dict:
    """
```

Gera resposta com citações inline.

Args:

pergunta: Pergunta do usuário
contextos: Documentos recuperados

Returns:

Resposta com citações

"""

```
contexto_numerado = "\n\n".join([
    f"[{i+1}] {c['texto']}"
    for i, c in enumerate(contextos)
])
```

```
prompt = f"""
```

Responda à pergunta usando as informações dos documentos.
Cite as fontes usando [número] após cada informação usada.

DOCUMENTOS:

```
{contexto_numerado}
```

PERGUNTA: {pergunta}


```

RESPOSTA (com citações [1], [2], etc.):
"""

response = self.client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": prompt}]
)

return {
    "resposta": response.choices[0].message.content,
    "fontes": [
        {"numero": i+1, "texto": c["texto"][:100] + "...", "fonte":
c["meta"].get("fonte", "N/A")}
        for i, c in enumerate(contextos)
    ]
}

def consultar(self, pergunta: str, historico: list = None) -> dict:
    """
    Consulta completa com todas as funcionalidades.
    """
    # Reformular se houver histórico
    pergunta_final = self.reformular_pergunta(pergunta, historico)

    # Recuperar com reranking
    contextos = self.recuperar_hibrido(pergunta_final, k=3)

    # Gerar resposta com citações
    resultado = self.gerar_com_citacoes(pergunta_final, contextos)

    return {
        "pergunta_original": pergunta,
        "pergunta_reformulada": pergunta_final,
        **resultado
    }

```

Avaliação de Sistemas RAG

Métricas Principais

A avaliação de sistemas RAG exige métricas que vão além da simples correção textual. Como a resposta é condicionada a documentos recuperados, é essencial medir tanto a **qualidade da recuperação** quanto a **fidelidade da geração**. Métricas como precisão e recall avaliam se os

documentos corretos foram recuperados, enquanto métricas como **faithfulness** verificam se o modelo respeitou estritamente o contexto fornecido, aspecto central para evitar alucinações. A relevância e a clareza completam o conjunto, garantindo que a resposta seja útil e compreensível ao usuário final.

Métrica	O que mede	Como calcular
Precisão	Relevância dos docs recuperados	Docs relevantes / Total recuperados
Recall	Cobertura dos docs relevantes	Docs relevantes recuperados / Total relevantes
Faithfulness	Fidelidade ao contexto	Resposta baseada apenas no contexto
Relevância	Resposta atende a pergunta	Avaliação humana ou LLM

No código abaixo, a função `avaliar_resposta_rag` utiliza um modelo de linguagem para realizar uma **avaliação automatizada da qualidade de uma resposta RAG**. A função compara a pergunta original, o contexto fornecido ao modelo e a resposta gerada, podendo opcionalmente considerar uma resposta esperada como referência. O modelo atribui notas normalizadas entre 0 e 1 para critérios como relevância, fidelidade ao contexto, completude e clareza, além de gerar comentários qualitativos.

Essa abordagem permite criar pipelines de avaliação contínua de sistemas RAG, viabilizando testes A/B, monitoramento de qualidade ao longo do tempo e auditoria de respostas em aplicações sensíveis, como sistemas jurídicos, de compliance e de atendimento ao cliente.

```
def avaliar_resposta_rag(pergunta: str, resposta: str,
                        contexto: str, resposta_esperada: str = None) ->
dict:
    """
    Avalia qualidade de uma resposta RAG.

    Args:
        pergunta: Pergunta original
        resposta: Resposta gerada
        contexto: Contexto usado
        resposta_esperada: Ground truth (opcional)

    Returns:
        Métricas de avaliação
    """
    prompt = f"""
    Avalie a resposta de um sistema RAG.
```

```
PERGUNTA: {pergunta}
CONTEXTO FORNECIDO: {contexto}
RESPOSTA GERADA: {resposta}
{"RESPOSTA ESPERADA: " + resposta_esperada if resposta_esperada else ""}
```

Avalie de 0 a 1:

1. relevancia: A resposta atende à pergunta?
2. faithfulness: A resposta usa apenas informações do contexto?
3. completude: A resposta cobre todos os aspectos importantes?
4. clareza: A resposta é clara e bem estruturada?

```
Retorne JSON: {"relevancia": 0.0, "faithfulness": 0.0, "completude":
0.0, "clareza": 0.0, "comentarios": ""}
"""
```

```
response = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": prompt}],
    response_format={"type": "json_object"}
)

return json.loads(response.choices[0].message.content)
```